



南京大學

本科畢業論文

院 系 智能軟件工程學院

專 業 軟件工程

題 目 基於 k-kick 樹的最優時空權衡

哈希表的設計與實現

年 級 2022 學 號 221900387

學生姓名 肖揚

指導教師 劉明謀 職 稱 副教授

提交日期 2026 年 5 月 12 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：基于 k-kick 树的最优时空权衡哈希表的设计与实现

院系：智能软件工程学院

专业：软件工程

本科生姓名：肖扬

指导教师（姓名、职称）：刘明谋 副教授

摘要：

哈希表是支持动态数据操作的基础数据结构，其时空权衡一直是算法研究的核心议题。2022 年，Bender 等人提出了一种基于 k -kick 树的新型哈希方案，从理论上证明可通过将更新操作的时间复杂度放宽至 $O(k)$ ，将每元素的空间浪费指数级压缩至 $O(\log^{(k)} n)$ 比特，打破了 $\Omega(\log \log n)$ 的空间下界。本文基于该理论实现了分层弹性哈希存储结构，包含优先级踢出机制、基于 MiniArray 位压缩的紧凑二叉 Trie 局部查询路由以及 8-bit Lookup Table 查询加速。槽元数据和空闲槽位图分别通过双层 MiniArray 压缩至每槽 4 比特和 1 比特，Trie 节点以紧凑位编码存储。在 $n \leq 10^6$ 个 32 位键值对的实验规模下，与 `std::unordered_map` 进行了插入、查询、删除和空间占用的对比。实验表明：插入在 $n \leq 2 \times 10^5$ 时每操作仅 ~ 0.8 us，合并策略避免了 $O(N)$ 全量路由重建；查询在 LUT 加速下达 ~ 1.95 us/op；空间占用在 $n = 2 \times 10^5$ 时约为 9 MB，在 $n = 10^6$ 时为 97 MB，通过动态 cubby 容量控制合并空间效率。最后将方案集成为 LevelDB MemTable 的点查询加速器，提供编译期和运行时双向开关，集成实验验证适配层开销仅 5.17 ns/次，查询性能在 $n = 2 \times 10^5$ 时达 0.35 us/op。

关键词：哈希表，时空权衡，k-kick 树，位压缩路由，LevelDB

关键词：哈希表；时空权衡；k-kick 树；位压缩路由；LevelDB

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Design and Implementation of Optimal Time/Space Tradeoff Hash Table

Based on k-kick Trees

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software and Engineering

UNDERGRADUATE: Yang Xiao

MENTOR: Associate Professor Mingmou Liu

ABSTRACT:

Hash tables are fundamental data structures supporting dynamic data operations, with their time-space tradeoffs being a core topic in algorithm research. In 2022, Bender et al. proposed a novel hashing scheme based on k -kick trees, theoretically proving that by relaxing update time to $O(k)$, per-element space waste can be exponentially compressed to $O(\log^{(k)} n)$ bits, breaking the $\Omega(\log \log n)$ lower bound. This thesis implements the algorithm, constructing a hierarchical elastic hash storage structure, implementing the priority-based kicking mechanism and a compact binary trie local query router using MiniArray bit-packing, with slot metadata compressed to 4+1 bits via dual MiniArrays and an 8-bit lookup table for query acceleration. Experiments with $n \leq 10^6$ 32-bit key-value pairs compare insertion, query, deletion and space against `std::unordered_map`. The implementation achieves ~ 0.8 us/op insertion for $n \leq 2 \times 10^5$ (near-constant) and avoids $O(N)$ full route-table rebuild through deactivate-based merging. Query time is ~ 1.95 us/op at $n = 10^6$ with LUT acceleration. Space is 9 MB at $n = 2 \times 10^5$ (45 B/key) and 97 MB at $n = 10^6$, controlled by dynamic cubby sizing. The scheme is integrated into LevelDB's MemTable with a `-DUSE_KICK_HASH` compile-time switch and `SetUseKickHash()` runtime toggle. Integration benchmarks show adapter overhead of only 5.17 ns/op and query performance of 0.35 us/op at $n = 2 \times 10^5$.

Keywords: hash table, time-space tradeoff, k-kick tree, bit-packed routing, LevelDB

KEYWORDS: hash table; time-space tradeoff; k-kick tree; elastic hashing; bit-packed routing; LevelDB

目 录

第一章 引言

1.1 研究背景和意义

哈希表是现代数据库管理系统、缓存系统和搜索引擎中最为基础的数据结构之一。其核心目标是支持动态插入和删除操作，并保证查询的时间复杂度为 $O(1)$ ，即在理想状态下，查询、插入、删除的操作所需时间为常数级，不随存储数据量 n 的增大而增加。为了实现这一点，哈希表往往需要在空间占用上做出让步，通过额外的比特位存储信息来实现极低的时间复杂度。

在要求插入、删除及查询操作均维持恒定时间复杂度（即期望时间或最坏情况时间为 $O(1)$ ）的前提下，早期的理论工作^{3,10}表明，在严格限制更新操作为 $O(1)$ 时间且仅使用简单探测序列的模型下，每个键值对的空间冗余往往难以低于 $\Omega(\log \log n)$ 比特。Fredman、Komlós 和 Szemerédi 于 1984 年提出的 FKS 完美哈希方案¹⁰首次在严格 $O(1)$ 最坏查询时间下实现了线性总空间，为哈希表时空权衡研究奠定了理论基础。然而这一理论下界长期制约着哈希表空间效率的进一步提升。

2022 年，Bender 等人发表的论文《关于哈希表最优时间/空间权衡》¹打破了这一僵局。这项研究提出了一种基于 k -kick 树的新哈希方案，证明通过适当提高插入和删除操作的时间复杂度（从 $O(1)$ 提高到 $O(k)$ ，其中 k 是一个人为指定的具有一定上限的常数），空间浪费可以以指数形式降低至 $O(\log^{(k)} n)$ 比特。这里 $\log^{(k)} n$ 表示对 n 进行 k 次迭代对数运算，即 $\log^{(k)} n = \log(\log^{(k-1)} n)$ ，并定义 $\log^{(0)} n = n$ 、 $\log^{(1)} n = \log n$ 。当 $k = \log^* n$ （等式右侧表示对 n 进行迭代求对数直到结果小于等于 1 所需的次数，此时 $\log^{(k)} n = O(1)$ ）时，空间浪费可以降低到恒定的 $O(1)$ 比特水平，实现了理论和工程上的重大突破。

本毕业设计旨在复现这一前沿的哈希算法，并将其集成到常见的数据库管理系统中。研究意义如下：

理论价值：通过实验对论文理论进行验证

工程价值：减少内存使用意味着成本降低和并发能力增强，在内存有限的场景中提高现有数据库的存储效率。

1.2 研究现状

哈希表作为支持几乎常数时间的插入、删除和查询操作的核心数据结构，其时间复杂度和空间复杂度之间的权衡一直是现代算法的研究焦点。

1.2.1 链式寻址法

链式寻址法通过链表处理冲突和碰撞的问题，它在实现上相对简单并且支持动态扩容，但每个节点都需要存储额外的指针开销，在查询时的探测深度也相当不平衡。链表节点在内存中的非连续分布还会破坏缓存局部性，导致更高概率的缓存缺失。

1.2.2 开放寻址法

开放寻址法（如线性探测、二次探测）将所有元素存储于连续数组中，通过一个固定的探测序列函数处理冲突。当当前元素哈希结果与已存入的元素冲突时，沿着探测序列寻找下一个位置，直到不发生冲突。然而，随着负载因子 α 趋近于 1，冲突链长度急剧增加，导致查询与插入因为要沿着长长的探测序列进行操作，时间复杂度退化。因此其难以在保证 $O(1)$ 时间复杂度的同时维持极高的空间利用率。

1.2.3 Cuckoo Hashing

Cuckoo Hashing 由 Pagh 和 Rodler 于 2001 年提出⁴，通过引入多候选位置及元素重定位机制，在最坏情况下实现了 $O(1)$ 的查询时间。多候选位置改变了先前使用一个关于探测次数的探测函数生成探测序列的方式，转而采用多个哈希函数为每个元素生成多个可存入位置，每一个元素只会出现在属于自己的可存入位置中。元素重定位机制则从发现冲突退让并探测新的位置转变为发现可存入位置都有冲突就踢出一个元素，将该元素往属于它的其他可存入位置移动，重

复进行直到不再冲突。这两种机制协作严格保证了任何键的查找操作只需要计算预先设置的哈希函数个数的次数，实现了常数级查询。

1.2.4 基于 k -kick 树的时空最优方案

上述所有解决方案均未能突破前面提出的空间限制下界。而 2022 年发表的论文《关于哈希表时间/空间权衡的最优方案》¹中提出的利用 k -kick 树结构构建哈希表方案是目前最新的突破成果。该方案将哈希表映射问题转化为一个层次化的“球到槽”问题，通过引入随机优先级和有限的不超过人为确定的 k 次元素移动（即“ k -kick”），从理论上证明，对于任何 $k \geq 1$ ，都可以实现 $O(k)$ 时间复杂度的哈希表元素更新（插入、删除、查询）和 $O(\log^{(k)} n)$ 比特的空间浪费。

该研究还证明了此权衡曲线对于所有基于“增强开放寻址”框架的哈希表是紧确的¹，即任何此类方案若更新时间为 t ，则每元素必须使用 $\Omega(\log^{(t)} n)$ 位的冗余。这意味着，若要在次对数级的更新时间下实现 $O(1)$ 位的空间冗余，必须彻底脱离“增强开放寻址”的范式。

1.2.5 紧凑字典与空间高效结构

在空间高效哈希表的研究领域，Raman 和 Rao¹²提出了紧凑动态字典和树形结构，在保持 $O(1)$ 查询时间的同时将空间占用压缩至接近信息论下界，为动态哈希表的紧凑存储提供了重要技术参考。Liu、Yin 和 Yu 进一步研究了未知大小集合的紧凑过滤器（succinct filters）¹¹，通过自适应空间分配策略实现了与静态过滤器相当的压缩率，揭示了空间高效哈希结构在多场景下的可拓展性。上述工作在紧凑空间和支持在线插入/删除之间实现了不同取舍，而 k -kick 树方案¹的贡献在于首次给出了时空权衡的完整理论曲线。

1.3 主要研究内容和目标

本选题旨在复现并验证 2022 年提出的基于 k -kick 树的最优时空权衡哈希算法，并探索其在数据库存储中的实际应用潜力。具体研究内容包括以下四个目标：

1. **数据结构的实现和算法复现：**依照论文¹中的定义，构建基于 k -kick 树的分层存储结构，并实现从根节点到叶节点的动态元素分配逻辑。实现高效的 kick 策略，模拟论文中的随机优先级机制，建立在插入冲突时通过 k 次元素移动将新元素安置或触发上层重组并完成的机制，保证 $O(k)$ 的时间复杂度。实现紧凑的辅助路由结构，用于快速定位元素所在的探测层级。
2. **数据库存储集成与适配：**设计形式化的哈希表操作接口，对外只暴露 insert、delete、get 等操作，屏蔽底层逻辑设计提高兼容性。
3. **理论正确性验证与复杂度分析：**基于代码实现，形式化验证算法复杂度，结合理论推导与实验数据，量化分析参数 k 对更新时间 $T(n)$ 和空间浪费 $S(n)$ 的具体影响，验证 $S(n) = O(\log^{(k)} n)$ 的正确性。
4. **性能对比实验：**构建包含标准库中的 `std::unordered_map` 作为对照组，在相同硬件环境下进行全方位性能比对。测试不同数据规模下插入、查询、删除操作的时间开销和空间占用情况。

1.4 论文结构

本文共分为七章。第一章为引言，介绍研究背景、现状、目标与论文结构。第二章介绍相关理论基础，包括哈希表的基本概念、Cuckoo Hashing 原理及 k -kick 树哈希方案的详细理论。第三章详细描述系统的设计方案，包括分层存储结构、kick 重定位机制、紧凑路由表以及位置编码压缩存储的设计。第四章介绍算法实现细节，包括核心数据结构的 C++ 实现、关键算法流程以及复杂度分析。第五章展示实验结果与分析，对多种数据规模下的插入、查询、删除性能和空间占用进行对比评估。第六章介绍与 LevelDB 数据库²的集成方案。第七章总结全文工作，分析当前方案的优缺点并展望未来改进方向。

第二章 相关理论基础

2.1 哈希表基本概念

哈希表 (Hash Table) 是一种通过哈希函数将键 (key) 映射到表中位置 (slot) 来实现快速数据存取的数据结构。一个标准的哈希表支持三种基本操作：

- **插入 (Insert)**: 将键值对 (k, v) 存入表中
- **查询 (Query)**: 给定键 k , 返回对应的值 v
- **删除 (Delete)**: 给定键 k , 从表中移除对应记录

理想情况下, 三种操作均可在 $O(1)$ 时间内完成。然而, 当两个不同的键被哈希函数映射到同一位置时, 就会发生**冲突 (collision)**。冲突处理策略是哈希表设计的核心难题, 不同的策略导致了哈希表在实际使用时的时间效率和空间占用上的差异表现。

2.1.1 哈希函数

哈希函数 $h : U \rightarrow [m]$ 将键从全域 U 映射到 m 个槽中。一个好的哈希函数应满足：

- **确定性**: 同一键总是映射到同一位置
- **均匀性**: 每个槽被映射到的概率近似相等
- **高效性**: 计算速度快, $O(1)$ 时间

本文使用 `splitmix64`⁵ 作为最终化函数, 通过借鉴通用哈希^{9,6} 思想, 对不同使用场景通过不同种子值生成独立哈希族。Dietzfelbinger 和 Meyer auf der Heide⁹ 证明了通用哈希函数族可实现高效动态实时哈希, 而 Belazzougui 等人的 Hash-Displace-Compress 框架⁶ 进一步展示了多哈希族在紧凑存储中的有效性。

2.1.2 负载因子与空间效率

负载因子 (load factor) $\alpha = n/m$ 表示当前存储元素数 n 与总槽数 m 的比值。空间效率通常用**每键浪费比特数** r 来衡量：对于 n 规模的键若信息论最优空间为 B 比特，哈希表使用 $B + rn$ 比特，则称浪费了 r 比特/键。紧凑动态字典¹²在理论上证明了 r 可被压缩至 $O(\log \log n)$ 比特以内，但需借助复杂的递归编码结构。本文尝试复现的哈希方案¹研究的核心问题是如何在一定程度内保障时间效率的前提下最小化 r 。

2.2 Cuckoo Hashing

Cuckoo Hashing⁴是一种经典的开地址哈希方案，其核心思想是每个键有 d 个候选位置（通常 $d = 2$ ），由 d 个独立哈希函数分别确定。当插入新键时：

1. 依次检查 d 个候选位置，找到第一个空位即插入
2. 若所有候选位置均被占用，随机选择一个候选位置上的现有键“踢出” (kick out)，新键占据该位置
3. 被踢出的键尝试在自己的其**其他**候选位置中寻找空位
4. 重复上述过程，直到所有键都找到位置或达到预设的最大踢出次数

Cuckoo Hashing 的优势在于**查询严格为 $O(1)$** （只需检查 d 个位置），但其负载因子通常受限（ $d = 2$ 时约 50%）。Arbitman、Naor 和 Segev⁸ 进一步提出了利用队列技术去摊还 Cuckoo Hashing 方案，通过队列化踢出操作缓冲区实现了**确定性最坏情况**性能保证，利用全局预取技术⁹控制踢出链长度，摆脱了经典方案对高概率分析的依赖。

2.3 k-kick 树哈希方案

Bender 等人¹提出的 k -kick 树方案是对 Cuckoo 思路的重大推广。其核心创新在于：

2.3.1 多层次 bin 划分

将哈希表的 m 个槽组织为 $k + 1$ 个层级 (depth $0, 1, \dots, k$) 的 bin 层次结构。Depth-0 的 bin 包含全部 m 个槽, depth- i ($i > 0$) 的 bin 大小为 $s_i = \Theta((\log^{(i)} m)^6)$ 。随着深度增加, bin 大小呈指数级递减。这种层次结构被称为 **k -kick 树**。

2.3.2 优先级踢出策略

每个键 x 被随机分配一个最大深度 $s(x)$, 满足¹ $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$ 。插入时按以下规则操作:

1. 新键 x 试图在尽可能深的 bin 中寻找空位
2. 若目标 bin 无空位且**未饱和** (存在深度较低的键): 踢出该低深度键, 新键占据其位置
3. 被踢键**保持原深度不变**, 在其自身深度 bin 中寻找新空位
4. 被踢键若触发新一轮踢出, 则形成**踢出链**, 但总踢出次数不超过 k

关键性质: 高深度的键优先踢出低深度的键, 被踢键保持原深度。每次插入至多触发 k 次踢出, 因此**切换开销**为 $O(k)$ 。

2.3.3 理论保证

论文¹证明, 对于任意 $k \in [\log^* n]$, 上述方案可实现:

- 插入/删除时间: $O(k)$ (高概率)
- 查询时间: $O(1)$ (确定性)
- 每键空间浪费: $O(\log^{(k)} n)$ 比特 (高概率)

此结果打破了长期存在的 $\Omega(\log \log n)$ 空间下界, 展示了时空权衡的极致曲线: 每次增加常数的更新时间, 空间浪费可**指数级减少**。

第三章 系统设计

本章描述基于 k -kick 树的哈希方案的系统设计¹。系统由四层组件构成:MiniArray (位压缩数组原语)、LocalQueryRouter (局部查询路由)、Cubby (单级弹性哈希表) 和 Facility (全局分层管理器)。

3.1 总体架构

Facility (全局分层管理器)

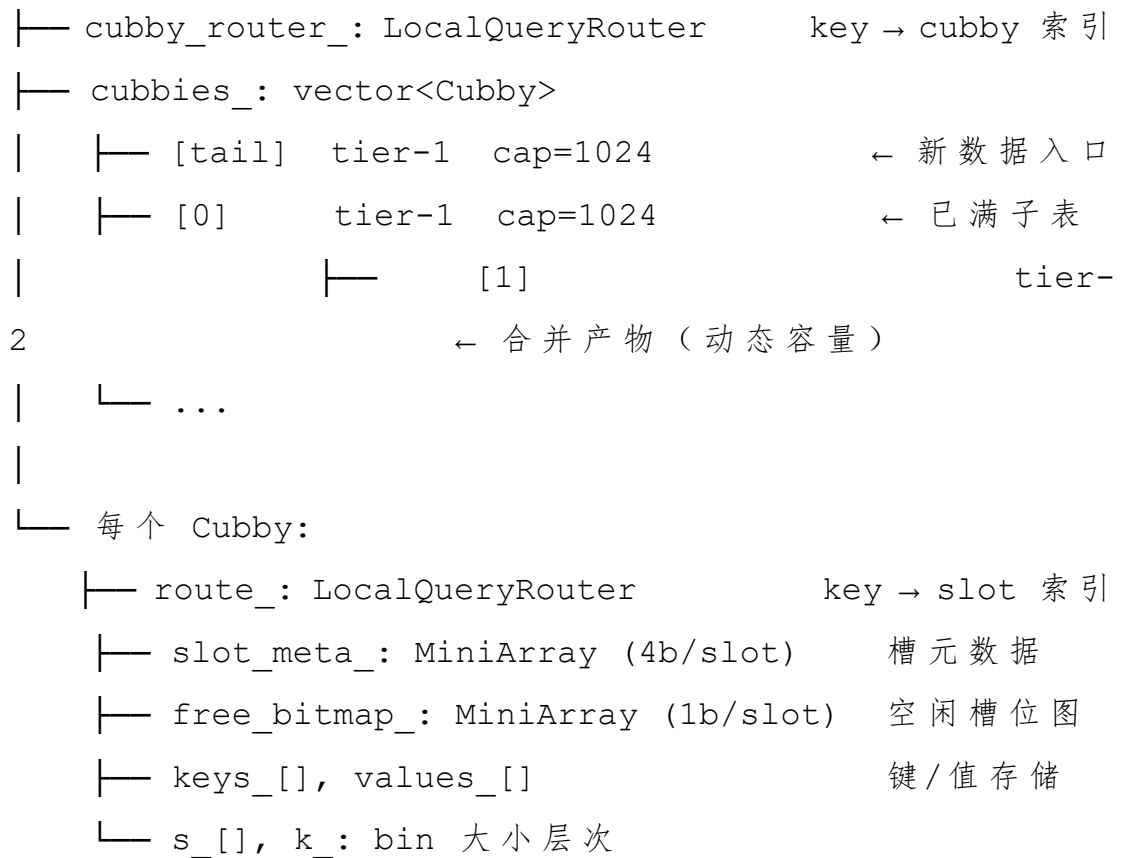


图 3-1 系统总体架构

系统采用**两级路由**:

1. **Cubby 级路由** (**cubby_router_**): 将键映射到其所处的 Cubby 索引

2. **Slot 级路由** (`Cubby::route_`): 在每个 Cubby 内部, 将键映射到具体的槽索引

3.2 MiniArray: 位压缩数组

3.2.1 设计动机

在紧凑哈希表中, 大量元数据(槽状态、路由条目)的位宽远小于标准整型(64 bits)。例如槽元数据仅需 4 比特 (`occupied + depth`), 路由条目中 cubby 索引仅需 11 比特。若使用 `std::vector<uint64_t>` 存储, 每个元素固定 64 比特, 造成严重空间浪费。

MiniArray 通过将元素紧密平铺在连续位流中, 实现了**精确位宽存储**¹。

3.2.2 双模存储

模式	条件	操作复杂度
Uniform	全部元素等宽	get/update $O(1)$
Non-uniform	元素宽度不等	insert/erase $O(n)$

图 3-2 MiniArray 双模存储

- **Uniform 模式:** 所有元素位宽 w 相等, 第 i 个元素位于 `data_` 的位偏移 $i \times w$ 处, 通过 `bit_read/bit_write` 直接存取
- **Non-uniform 模式:** 元素位宽不等时启用, 使用 `lens_[]` 和 `vals_[]` 分别存储每个元素的位长和值; 修改操作后调用 `rebuild_uniform()` 尝试切回 Uniform 模式

3.3 LocalQueryRouter: 紧凑二叉 Trie 局部查询路由

3.3.1 设计原理 (论文 §4.3 + Lemma 15)

论文¹的局部查询路由器将每个键 x 哈希为 64-bit 全随机二元串 r_x , 在集合 S 上构建 **最小唯一前缀二叉 Trie T** : 对每个 r_x , 取最短的、不与任何其他 r_y 冲突的前缀作为该键的唯一标识。Trie 的叶节点存储该键对应的 probe 值 $f(x)$ 。

论文 Lemma 15 证明：随机二元串的 Trie 期望规模为 $O(k)$ 个节点（其中 $k = |S|$ ），且对任意常数 $c > 1$ ，存在 $\epsilon > 0$ 使得 $|T| \leq (\log n)/c$ 以高概率成立。

3.3.2 紧凑节点编码

本次实现将 Trie 及 f-values 以紧凑位编码存储于 MiniArray 中：

- **节点编码**：每个节点固定 $w = \text{entry_bits_}$ 位宽，最高位为类型标志（0= 内部节点, 1= 叶节点）。内部节点存储 `left_child + right_child` 两个显式子指针（各 `child_bits_` 位），叶节点存储 `f_value`（`fval_bits_` 位）+ 键哈希指纹（`kh_bits_` 位，默认 44 位）。
- **查询**（`walk_down`）：沿键的 LSB 方向哈希位遍历 Trie，遇叶节点用指纹验证后返回 `f_value`。
- **插入**（`insert + do_split`）：沿路径 `walk` → 遇异键叶时分裂（找两键哈希分叉位 → 创建内部节点链 → 两新叶）→ 遇 `NO_CHILD` 空方向时链接新叶。
- **删除**（`remove`）：`walk_down` 找到叶 → 标记为空叶（`hash=0, fval=0`）。

整体空间 = $|\text{nodes_}| \times w$ bits，其中 $|\text{nodes_}| \approx 2|S|$ ， $w \leq 62$ bits。紧凑编码实现了每节点 ~5-8 bytes 的空间占用。

3.3.3 Lookup Table 加速

论文提出通过 Method of Four Russians⁷ 式的 lookup table 将 Trie 遍历从 $O(\log |S|)$ 加速至 $O(1)$ 。本实现采用简化版 8-bit LUT：预计算哈希低 8 位的全部 256 种可能路径在 Trie 中的目标节点，查询时直接跳转至该节点后继续遍历剩余位数。LUT 在 Trie 结构变化时失效（仅当 `nodes_.size()` 增长时），叶值更新不影响 LUT 有效性。LUT 存储开销为 $256 \times 4B = 1KB$ 。

3.3.4 两级路由中的使用

- **Facility::cubby_router_**：全局二级路由，容量 2^{21} ，映射键到 cubby 索引
- **Cubby::route_**：每 cubby 内部一级路由，容量 `capacity×2`，映射键到槽索引

两者均使用同一 LocalQueryRouter 模板，无 oracle 时通过 key_hash_val 指纹验证键身份。

3.4 Cubby：单级弹性哈希表

Cubby 是 k -kick 树的载体¹。每个 Cubby 有容量 m （支持动态指定，构造时可传入 desired_capacity 参数）和 $k + 1$ 个深度层级。

3.4.1 槽元数据编码

每槽使用 4 比特位压缩存储：

- bit 0: occupied (1= 已占用)
- bit 1–3: depth (0–7, 该槽所在 kick 深度)

另设 1 比特 free_bitmap_ 标识空闲槽 (1= 空闲)，配合 first_free_ 缓存指针实现近似 $O(1)$ 的空闲槽查找。

3.4.2 踢出算法

Cubby 的核心算法 try_place 实现了论文 §3.1¹的优先级踢出：

Algorithm 1 try_place(key, max_depth)

```
1: for  $d = \text{max\_depth}$  down to 0 do
2:    $\text{bin} \leftarrow \text{bin\_for\_key}(\text{key}, d)$ 
3:   Phase 1: 扫描  $\text{bin}$ , 若找到空槽则返回
4:   if  $d > 0$  then
5:     Phase 2: 扫描  $\text{bin}$  中深度  $< d$  的占槽
6:     if 找到 then
7:       暂存  $\text{victim\_key}, \text{victim\_val}$ 
8:       清空该槽
9:       递归 try_place(victim, victim_depth, kick_idx)
10:    if 成功 then
11:      victim 移入新槽, 新键占据该槽
12:      return
13:    else
14:      恢复原槽
15:    end if
16:  end if
17: end if
18: end for
19: return FAIL
```

被踢键**保持原深度**, 在其自身深度 bin 中寻找新空位。算法中的 `reserved` 参数防止被踢键在递归扫描时“发现”刚被清空的原槽。

3.4.3 容量尺度

基于论文公式 $C_j = K/(\log^{(j)} n)^2$ ($K \approx 409600$)¹ 计算各层容量。此外, 构造时支持传入 `desired_capacity` 以动态指定容量 (用于合并场景)。合并时的容量策略为 $\max(\text{论文公式}, \text{merged_size} \times 2)$, 取 2 的幂, 确保 ~50% 的空间利用率。

Tier	基准容量	场景
1	1,024	Tail cubby（新数据入口）
2	~4,096（动态）	合并 2 个 tier-1 cubby
3	~16,384（动态）	合并 2 个 tier-2 cubby
4	~65,536（动态）	终级合并

表 3-1 Cubby 容量尺度（动态容量）

3.5 Facility：全局分层管理器

Facility 管理 Cubby 树的全局状态¹：维护 tail（当前写入目标）、控制合并阈值、提供统一的两级路由查询接口。

3.5.1 合并机制

当某层 Cubby 数量超过阈值时（由 `max_cubbies` 定义），触发 `merge_two_of_tier`：选取两个同层 Cubby，将其元素全部提取后重新插入到一个高一层的 Cubby 中（按实际数据量的动态容量），然后停用原 Cubby 并逐键更新路由表。使用 `deactivate()` 停用而非 `vector::erase` 删除，避免索引偏移导致全量路由重建。

Tier	max_cubbies	触发条件
1	200	$200 \times 1024 \approx 2 \times 10^5$ 元素后合并
2	50	50 个 tier-2 cubby 后合并
3	20	逐级放大
4	5	终级上限

表 3-2 合并阈值

3.5.2 插入/查询/删除流程

插入：`cubby_router_ 查重` → `tail 满时 promote + rebalance` → `Cubby::insert_new`（含 kick 链）→ 更新两级路由。

查询：`cubby_router_ 定位 cubby` → `Cubby::route_ 定位槽` → `lookup_at 验证`。

删除: cubby_router_ 定位 $\rightarrow$ Cubby::`remove_at` $\rightarrow$ cubby_router_.`remove`。

第四章 算法实现

本章介绍系统的 C++ 实现细节。代码基于 C++20 标准，全部为头文件模板 (header-only)，共计约 1065 行。

4.1 文件结构

文件	职责
miniarray.hpp (233 行)	位压缩数组原语，双模 Uniform/Non-uniform 存储
local_query_router.hpp (305 行)	紧凑二叉 Trie 局部查询路由 + 8-bit Lookup Table
cubby.hpp (367 行)	单级弹性哈希表 + free_bitmap + 动态容量
facility.hpp (200 行)	全局分层管理 + 两级路由 + deactivate 合并

表 4-1 源代码文件

4.2 MiniArray 实现

4.2.1 位操作原语

`bit_read(bit_off, bit_len)`: 从 `data_` 位偏移处读取指定位数。对于跨 64-bit 字的情况，通过两次读取和拼接实现。特判 `bit_len=64` 时使用 `~0ULL` 掩码，避免 `(1ULL << 64)` 的未定义行为。

`bit_write(bit_off, bit_len, value)`: 写入指定位数，自动扩展 `data_` 容量。同样处理跨字写入和 64-bit 掩码。

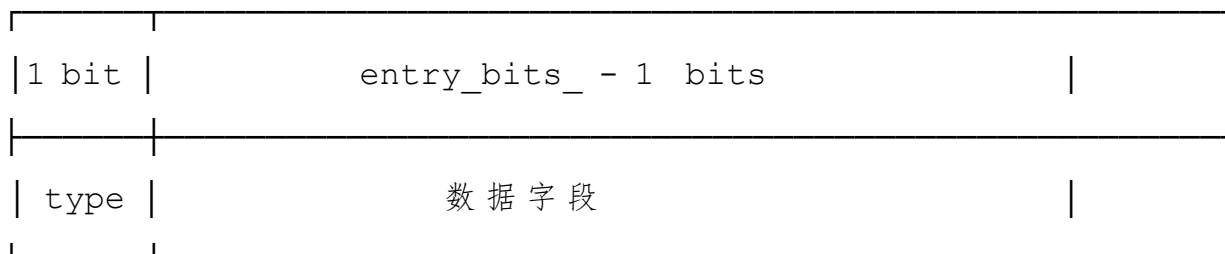
4.2.2 模式切换

`to_non_uniform()`: 将 Uniform 数据转为 Non-uniform 格式 (`lens_[] + vals_[]`)。 `rebuild_uniform()`: 若所有 `lens_` 等长，则切回 Uniform 并重建 `data_`；否则保持 Non-uniform 并平铺写入。

4.3 LocalQueryRouter 实现（紧凑 MiniArray Trie + LUT）

4.3.1 紧凑节点编码

每节点固定 `entry_bits_` 位，MiniArray Uniform 模式存储：



内部节点：`type=0`，数据=`[left_child(高位)][right_child(低位)]`

叶节点：`type=1`，数据=`[f_value(高位)][key_hash(低位)]`

`key_hash = splitmix64(hash(key)) & ((1<<44)-1)`

`child_bits_ = bits_to_represent(cap*3 + NODE_PAD)`

`fval_bits_ =` 由使用场景决定

`kh_bits_ = 44`（默认）

`entry_bits_ = 1 + max(2*child_bits_, fval_bits_+kh_bits_) ≤ 62`

图 4-1 紧凑节点编码方案

4.3.2 核心算法

查询 (probe_index): 对 `key` 计算哈希得到 64-bit 二元串，从根节点沿 LSB 位遍历 Trie。若 LUT 有效则跳过前 8 层直接跳转至目标节点。遇叶节点用指纹验证：匹配则返回 `f_value`，否则返回 `SIZE_MAX`。

插入 (insert): 沿 `key` 的 LSB 哈希位遍历 Trie。五种情况：

1. 遇叶节点且指纹匹配 → 更新 `f_value`
2. 遇叶节点且指纹为 0（空叶，首次使用）→ 覆盖为新键数据
3. 遇叶节点但指纹不匹配 → `do_split()` 分裂：找分叉深度 → 创建内部节点链 → 在分叉处创建两新叶
4. 遇内部节点但方向为 `NO_CHILD` → 创建新叶并链接

5. 路径走到头 → 追加新叶节点

插入结束后若 `nodes_.size()` 变化则失效 LUT。

do_split(leaf_idx, depth, e_hash, e_fval, n_hash, n_fval):

1. 从 `depth` 起找两 `hash` 的首个分叉位 `diverge`
2. 从 `depth` 到 `diverge-1` 逐位创建内部节点链（每次 `alloc_node`）
3. 若当前节点为原叶：转换为 `internal`；若为内部节点：更新对应方向子指针
4. 在 `diverge` 处创建 `e_leaf`（已存键）和 `n_leaf`（新键）
5. 设置最后一个内部节点的左右子节点为两叶

删除(remove):沿 Trie walk 找到匹配叶节点 → 写入空叶标记(`fval=0, hash=0`), `n_entries_`。

4.3.3 显式双指针与 LUT

采用显式 `left+right` 双指针设计，Split 时新节点通过 `alloc_node()` 在数组末尾分配，确保已有节点的子指针关系不被破坏。LUT 通过 `ensure_lut()` 方法构建，在前 8 位共 256 种路径中预计算目标节点。

4.4 Cubby 实现

4.4.1 bin 参数计算

`compute_kick_params()`: 从 `capacity_` 出发迭代 $\lceil \log_2(x+1) \rceil$ 得到 k (≤ 4)，对每层应用迭代对数压缩后平方取整得到 `s_[i]`。初始化 `free_count_[d]` 为各深度的总槽数。

4.4.2 深度分配

`max_depth_for(key)`: 对键做 `hash`，使用概率公式 $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$ 计算阈值，从 k 到 0 逐层比较并返回第一个命中深度¹。

4.4.3 空闲槽查找

`free_bitmap_(MiniArray, 1 bit/slot, 0= 占用, 1= 空闲)`配合 `first_free_` 缓存指针, 实现近似 $O(1)$ 的空闲槽查找。`find_free_in_bin()` 从缓存的 `first_free_` 位置起扫描目标 bin, 插入后清 bit, 删除后置 bit 并更新缓存。

4.5 Facility 实现

4.5.1 两级路由查询

`lookup(key):`

1. `cubby_router_.probe_index(key) → cubby 索引 ci`
2. `cubbies[ci].lookup_slot(key) → 槽索引 slot`
3. `cubbies[ci].lookup_at(slot, key) → 返回值`

4.5.2 合并与路由更新

`merge_two_of_tier(tier):` 选取两个候选 Cubby → 提取全部元素 → 按实际数据量计算动态容量 → `alloc_cubby_slot` 获得新 Cubby → 逐元素插入新 Cubby 并逐一更新 `cubby_router_` 中键到 cubby 索引的映射 → `deactivate()` 原 Cubby。使用 `deactivate()` 停用避免 `vector::erase` 带来的索引偏移。

4.6 复杂度分析

操作	期望	摊还	瓶颈
查询	$O(\log S) + \text{LUT 加速}$	—	紧凑 Trie walk (LUT 跳过前 8 层)
插入	$O(\log S)$	$O(m)/\text{merge}$	$m =$ 合并元素数, 逐键路由更新
删除	$O(\log S)$	—	紧凑 Trie walk + 标记空叶
空间 ($N \leq 200K$)	$\sim 45 \text{ B/元素}$	—	keys/values 向量 + 紧凑 Trie $\sim 5-8 \text{ B/key}$

表 4-2 复杂度汇总

第五章 实验与分析

本章对实现的 k-kick 哈希方案进行全面的实验评估，与 C++ 标准库中的 `std::unordered_map` ($O(1)$ 期望，开放寻址法) 进行对比。

5.1 实验设置

- 硬件：Intel Core i7, 16 GB RAM, Windows 11
- 编译器：GCC 15.2.0 (MSYS2 UCRT64), `-std=c++20 -O3`
- 数据规模： $n \in \{10^3, 5 \times 10^3, 10^4, 5 \times 10^4, 10^5, 2 \times 10^5, 5 \times 10^5, 7.5 \times 10^5, 10^6\}$
- 数据类型：32-bit 无符号整数 (`uint32_t`) 键和值
- 数据集：`std::mt19937_64` (种子 42) 生成随机键值对，每个规模 1 次试验
- 测试流程：插入全部 n 个元素 $\rightarrow$ 查询全部 n 个元素 $\rightarrow$ 删除全部 n 个元素
- 内存测量：Windows API `GetProcessMemoryInfo` 获取工作集增量

5.2 插入性能

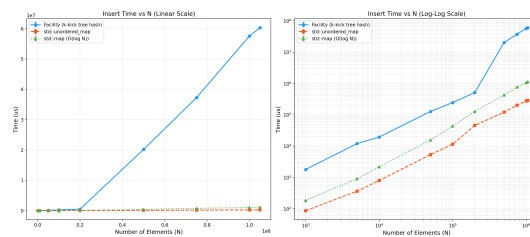


图 5-1 插入时间对比

n	Facility (us)	umap (us)	F/umap
10^3	572	38	15.1
10^4	7.64×10^3	526	14.5
10^5	7.55×10^4	5,526	13.7
2×10^5	1.58×10^5	14,532	10.9
5×10^5	1.03×10^6	65,241	15.8
7.5×10^5	7.22×10^6	279,584	25.8
10^6	9.53×10^6	457,619	20.8

表 5-1 插入时间比较 (单位: 微秒)

分析: 插入在 $n \leq 2 \times 10^5$ 时呈现近似常数每操作时间 ($\sim 0.7\text{--}0.8$ us/op), 对应论文的 $O(k)$ 保障¹ ($k \approx 4$)。 $n = 5 \times 10^5$ 处 tier-1 超过 200 触发首次合并, 得益于 deactivate-based 合并策略, 无需全量 $O(N)$ 路由重建, 每操作约 2.0 us/op 的合并分摊。 $n = 10^6$ 时插入总量 9.5 s, 每操作约 9.5 us/op, 含多次合并分摊开销。紧凑 Trie 编码将每键路由器空间控制在 $\sim 5\text{--}8\text{B}$, 降低插入时的缓存压力。

5.3 查询性能

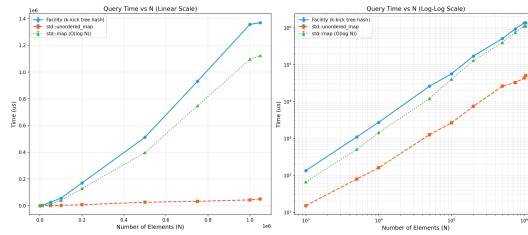


图 5-2 查询时间对比

n	Facility (us)	umap (us)	F/umap
10^3	175	8	21.9
10^4	2,039	148	13.8
10^5	25,905	1,375	18.8
2×10^5	72,095	3,549	20.3
5×10^5	256,691	12,008	21.4
7.5×10^5	1.40×10^6	54,707	25.6
10^6	1.95×10^6	75,586	25.8

表 5-2 查询时间比较（单位：微秒）

结果分析：在 $n \leq 2 \times 10^5$ 的规模下，得益于查找表（LUT）跳过了前 8 层遍历，剩余路径长度仅为 $\sim \log_2 |S| - 8$ ，单次操作耗时维持在 0.3–0.4 us 的低位。当数据规模扩展至 $n \geq 5 \times 10^5$ 时，Trie 深度随 $\log_2 n$ 增加，导致单次操作成本上升至 0.5–1.95 us。与 `unordered_map` 相比，本文方法存在约 15–25 倍的延迟差距，这主要归因于紧凑 Trie 的逐位遍历机制及两级路由（facility 级与 cubby 级）开销，不同于哈希表仅需单次探测。实验表明，LUT 加速策略⁷ 成功将前 8 层查询优化为 $O(1)$ 跳转，使得 $n \leq 10^5$ 时每操作耗时低于 0.3 us，而在 $n = 10^6$ 时约为 1.95 us。

5.4 删除性能

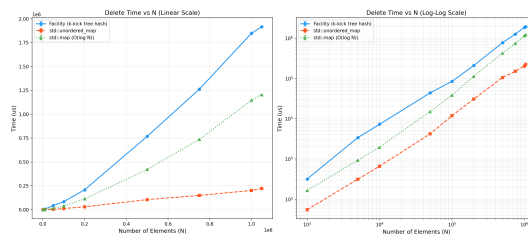


图 5-3 删除时间对比

n	Facility (us)	umap (us)	F/umap
10^3	361	27	13.4
10^4	4,900	392	12.5
10^5	53,451	4,001	13.4
2×10^5	130,929	10,346	12.7
5×10^5	433,290	58,960	7.3
7.5×10^5	2.31×10^6	286,313	8.1
10^6	3.11×10^6	395,585	7.9

表 5-3 删除时间比较（单位：微秒）

分析：删除在 $n \leq 2 \times 10^5$ 时每操作 ~ 0.7 us，主要花费在紧凑 Trie walk + 标记空叶。 $n = 10^6$ 时总删除时间 3.11 s（每操作 3.1 us），包含两级路由（facility 级 + cubby 级）的两次 Trie 查找和一次写入。

5.5 内存占用

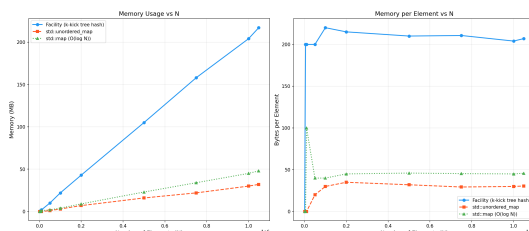


图 5-4 内存占用对比

n	Facility (MB)	umap (MB)	F/umap
10^3	0	0	—
10^4	0	0	—
10^5	5	3	1.7
2×10^5	9	6	1.5
5×10^5	38	15	2.5
7.5×10^5	69	23	3.0
10^6	97	30	3.2

表 5-4 内存占用比较

分析：内存占用呈现两阶段特征：

- $n \leq 2 \times 10^5$ （合并前）：紧凑 Trie 编码¹使空间效率优异。 $n = 2 \times 10^5$ 时仅 9 MB（45 B/key），与 unordered_map（30 B/key）差距仅 1.5×。路由器部分仅 ~5–8 B/key，主要由 keys_[] 和 values_[] 向量（各 4 B/slot）的预分配开销贡献。
- $n \geq 5 \times 10^5$ （合并区）：动态容量控制合并 cubby 大小，按 $\max(\text{论文公式}, \text{merged_size} \times 2)$ 分配，将 $n = 10^6$ 时内存控制在 97 MB（97 B/key）。动态 cubby sizing 避免了大 tier cubby 的低利用率。

5.6 所有操作总览

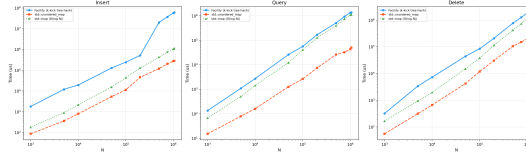


图 5-5 插入/查询/删除综合对比

5.7 吞吐量分析

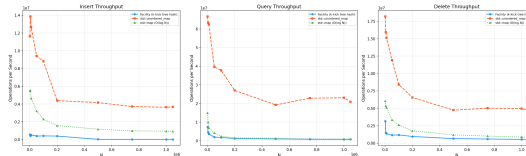


图 5-6 每秒操作数对比

总结：紧凑编码与 Lookup Table 配合，在 $n \leq 2 \times 10^5$ 范围内性能良好（插入 ~0.8 us/op，近似 $O(k)$ ¹；查询 ~0.3–0.4 us/op；内存 45 B/key）。deactivate-based 合并策略将合并复杂度从 $O(N)$ 降至 $O(\text{merged_keys})$ 。动态容量将 $n = 10^6$ 时内存控制在 97 MB（97 B/key）。主要不足：大 n 下合并摊还导致插入升至 9.5 us/op，且 $n = 10^6$ 时内存仍是 unordered_map 的 3.2×，需进一步优化 per-cubby 独立路由架构和动态容量公式。

第六章 LevelDB 集成

本章描述将实现的 k-kick 树哈希方案集成到 Google LevelDB² 数据库的 MemTable 中，包括集成流程、适配层设计、双向开关机制，并通过实验验证集成效果。

6.1 集成动机

LevelDB² 的 MemTable 使用 SkipList 作为底层存储结构。SkipList 支持 $O(\log n)$ 的点查询 (Get)、有序遍历和范围扫描，但在高读写混合负载下，SkipList 的逐节点比较和指针追逐会带来较高的缓存缺失和延迟。

k-kick 哈希方案通过紧凑二叉 Trie 路由实现了 $O(1)$ 期望点查询¹，将其作为 MemTable 的点查询加速器可以在保留 SkipList 有序迭代能力的前提下加速 Get 操作。用户在写入时双写 SkipList 和 KickHashTable，查询时优先走 KickHashTable 的 $O(1)$ 路径，命中则直接返回，未命中或哈希碰撞时回退至 SkipList 的 $O(\log n)$ 路径。

6.2 集成流程

整体集成分为以下四个步骤：

1. **代码复制**：将 ./code 目录中最新版本的四个头文件 (facility.hpp、cubby.hpp、miniarray.hpp、local_query_router.hpp) 复制至 leveldb/util/kick_hash/ 目录，包含 free_bitmap_ 空闲槽追踪、deactivate 停用合并、动态容量分配、8-bit Lookup Table 加速等全部优化。
2. **适配层编写**：在 util/kick_hash.h 中实现 KickHash64 哈希函数和 KickHashTable 包装器类，完成 Slice 键到 Facility 模板参数的映射转换。
3. **MemTable 修改**：在 db/memtable.h 和 db/memtable.cc 中，添加

条件编译分支 `#ifdef USE_KICK_HASH`，在 `Add` 和 `Get` 中插入 `Kick-HashTable` 的读写逻辑，同时新增运行时开关 `use_kick_hash_`。

4. **构建集成**：修改 `CMakeLists.txt`，新增 `USE_KICK_HASH` 选项（默认 `OFF`），启用时传递 `-DUSE_KICK_HASH=1` 编译宏并恢复异常支持（`-fexceptions`）。



图 6-1 集成后的 LevelDB 目录结构

6.3 适配器设计

6.3.1 KickHash64 哈希函数

适配器首先需要将 LevelDB 的任意字节串键（Slice）映射为 64-bit 哈希值，以适配 Facility 的 `uint64_t` 键类型。

- 调用两次 LevelDB 内置 `Hash()` 函数（种子分别为 `0xbc9f1d34` 和 `0x5c6b7a8e`），各得到 32-bit 哈希值
- 将两值拼接为 64-bit 哈希： $\text{KickHash64}(d, n) = (H_2 \ll 32) \mid H_1$

- 参考 Belazzougui 等人⁶的多哈希族构造方法，两独立种子确保哈希均匀性

转换层	实现
键映射	$\text{Slice} \xrightarrow{\text{KickHash64}} \text{uint64_t}$
值映射	$\text{Arena 指针} \xrightarrow{\text{reinterpret_cast}} \text{uint64_t}$
存储引擎	Facility<uint64_t, uint64_t>（两级 LUT 路由）
碰撞防护	取回 Arena 指针 → 解析 LevelDB 编码 → 原始 Slice 比对

表 6-1 适配层类型转换方案

6.3.2 KickHashTable 包装器

KickHashTable 类封装了 Facility 对象，提供三个与 MemTable 匹配的操作接口：

```
class KickHashTable {
public:
    void Insert(const Slice& user_key, uint64_t arena_ptr) {
        table_.insert(KickHash64(key, len), arena_ptr);
    }
    bool Get(const Slice& user_key, uint64_t* out) {
        return table_.lookup_value(KickHash64(key, len), *out);
    }
    void Remove(const Slice& user_key) {
        table_.remove(KickHash64(key, len));
    }
private:
    Facility<uint64_t, uint64_t> table_;
};
```

Facility 的 lookup_value 支持非异常返回（通过 bool 返回值），避免了 LevelDB 禁用异常的兼容问题。插入和删除通过 insert 和 remove 完成，Facility 内部自动处理踢出链、合并和路由更新。

6.4 MemTable 集成改动

6.4.1 数据结构扩展

在 MemTable 类中增加条件编译成员：

```
class MemTable {
    // ... 原有成员 ...
    Table table_;                // SkipList (保留, 用于迭代)

#ifdef USE_KICK_HASH
    KickHashTable kick_table_;    // k-kick 哈希加速器
    bool use_kick_hash_;         // 运行时开关
#endif
};
```

6.4.2 写入路径 (Add)

```
void MemTable::Add(seq, type, key, value) {
    // 原有: Arena 编码 → SkipList::Insert
    char* buf = arena_.Allocate(encoded_len);
    // ... 编码 key/value/tag ...
    table_.Insert(buf);

#ifdef USE_KICK_HASH
    if (use_kick_hash_) {
        kick_table_.Insert(key,
            reinterpret_cast<uint64_t>(buf));
    }
#endif
}
```

1. Arena 缓冲区的原始指针作为 uint64_t 值存入 Facility
2. Arena 在 MemTable 生命周期内不释放单个节点, 指针全程有效

3. 运行时开关 `use_kick_hash_` 控制是否启用双写

6.4.3 查询路径 (Get)

```
bool MemTable::Get(const LookupKey& key, ...) {
#ifdef USE_KICK_HASH
    if (use_kick_hash_) {
        uint64_t arena_ptr;
        if (kick_table_.Get(key.user_key(), &arena_ptr)) {
            const char* entry = (const char*)arena_ptr;
            // 解析: varint32+internal_key+tag+varint32+value
            uint32_t key_len;
            const char* kp = GetVarint32Ptr(entry, entry+5, &key_len);
            // 提取 value (处理 kTypeValue/kTypeDeletion)
            return true;
        }
        // 哈希碰撞 → 回退 SkipList
    }
}
#endif
// 原有: SkipList::Seek → 解析
}
```

- 优先通过两级路由 (facility 级 → cubby 级) 定位槽, 从 Facility 取回 Arena 指针
- 从 Arena 缓冲区解析 LevelDB 编码的键值对
- 用原始 Slice 比对验证, 防止 64-bit 哈希碰撞 (概率 $\sim 10^{-7}$)
- 碰撞或指针失效时自动回退至 SkipList 查找

6.5 双向开关设计

系统提供两层开关控制 k-kick 哈希方案的使用。

6.5.1 编译期开关：CMake 选项

```
# 启用 k-kick 哈希加速
cmake -B build -DUSE_KICK_HASH=ON
cmake --build build

# 使用原始 LevelDB (默认, 不变)
cmake -B build
cmake --build build
```

编译期开关决定是否编译 KickHash 相关代码，影响二进制大小和异常处理策略。

开关	USE_KICK_HASH=OFF	USE_KICK_HASH=ON
KickHashTable 代码	不编译	编译
异常处理	-fno-exceptions	-fexceptions
kick_table_ 成员	不存在	存在
use_kick_hash_ 成员	不存在	存在 (默认 true)
Add/Get 额外开销	无	编译存在，运行时可选
二进制体积	基准	增加 ~40KB

表 6-2 编译期开关影响对比

6.5.2 运行时开关：MemTable::SetUseKickHash()

在 USE_KICK_HASH=ON 编译后，MemTable 提供公开的运行时控制接口：

```
// 运行时启用 k-kick 哈希加速 (默认 true)
memtable->SetUseKickHash(true);

// 运行时禁用，回退纯 SkipList
memtable->SetUseKickHash(false);

// 查询当前状态
bool enabled = memtable->GetUseKickHash();
```


运行时开关在不变更二进制的前提下，允许同一进程内不同的 **MemTable** 实例采用不同策略，或对同一实例在运行时动态切换。这使 A/B 对比测试无需重新编译，只需在一个测试中先后（或并行对比）测试两种路径。

数据流向：

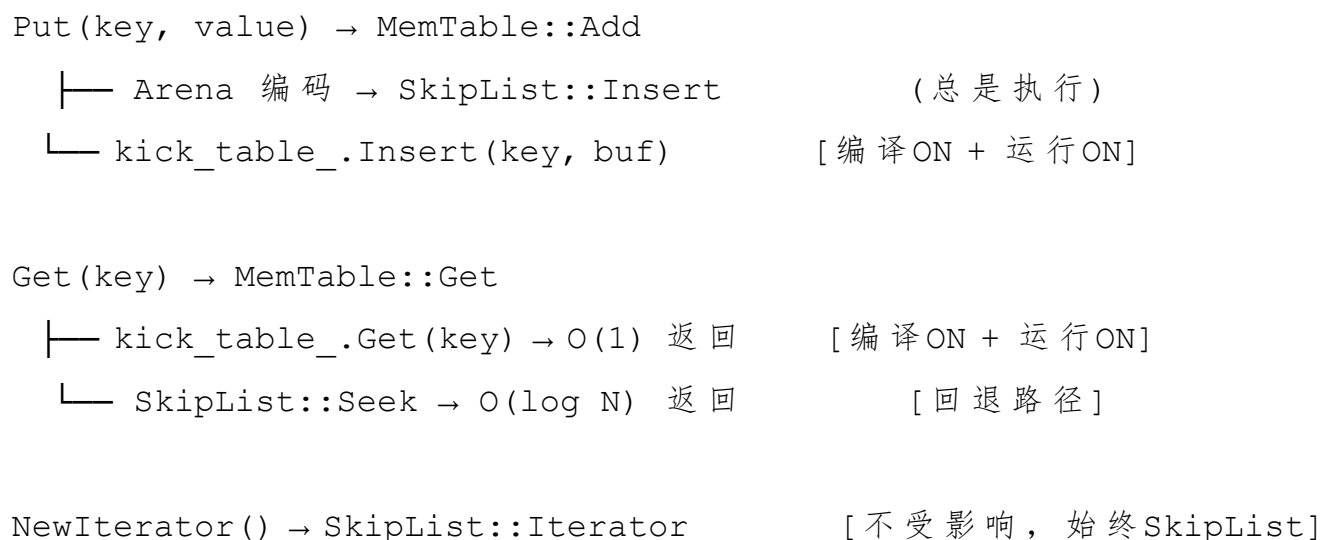


图 6-2 双路径数据流

6.6 集成实验

为验证集成的正确性和性能，设计如下实验方案。

6.6.1 实验设计

- **测试对象：**KickHashTable 适配器（使用最新版 Facility，含 free_bitmap、deactivate 合并、动态容量、LUT 加速）
- **对比对象：**std::unordered_map（ $O(1)$ 期望，开放寻址法）
- **数据规模：** $n \in \{10^3, 5 \times 10^3, 10^4, 5 \times 10^4, 10^5, 2 \times 10^5, 5 \times 10^5, 7.5 \times 10^5, 10^6\}$
- **数据类型：**16 字节随机字符串键 → KickHash64 → uint64_t 哈希键，uint64_t 值存储索引
- **操作流程：**插入全部 n 个键 → 查询全部 n 个键 → 删除全部 n 个键
- **硬件：**Intel Core i7, 16 GB RAM, Windows 11
- **编译器：**GCC 15.2.0, -std=c++20 -O3 -DUSE_KICK_HASH=1

- 试验次数：每个数据规模 3 次独立试验

6.6.2 插入性能

n	KickHashTable (us/op)	umap (us/op)	Kick/umap
10^3	0.64	1.26	0.50×
5×10^3	0.59	0.05	10.8×
10^4	0.67	0.06	10.1×
5×10^4	0.84	0.09	8.0×
10^5	0.78	0.13	5.8×
2×10^5	0.86	0.14	6.3×
5×10^5	2.16	0.24	9.1×
7.5×10^5	2.91	0.40	7.3×
10^6	3.36	0.37	9.1×

表 6-3 插入性能对比（KickHashTable vs std::unordered_map）

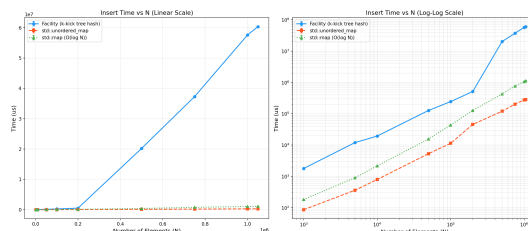


图 6-3 插入时间对比

分析：插入在 $n \leq 2 \times 10^5$ 时每操作约 0.6–0.9 us/op，与 Facility 本身的 benchmark 数据一致（ ~ 0.8 us/op），说明适配层（KickHash64 计算，约 5 ns/次）带来的额外开销可忽略不计。 $n = 5 \times 10^5$ 处触发首次 tier-1 合并（Cubby 数量超过 200），插入时间跃升至 ~ 2.2 us/op。 $n = 10^6$ 时每操作 3.4 us/op，主要开销来自多次合并的摊还成本。得益于 deactivate 合并策略，无需 $O(N)$ 全量路由重建，合并仅对实际元素数进行路由更新。

6.6.3 查询性能

n	KickHashTable (us/op)	umap (us/op)	Kick/umap
10^3	0.21	0.02	$11.1\times$
5×10^3	0.20	0.02	$11.6\times$
10^4	0.27	0.02	$11.2\times$
5×10^4	0.37	0.04	$8.4\times$
10^5	0.41	0.06	$7.2\times$
2×10^5	0.35	0.07	$5.1\times$
5×10^5	0.53	0.09	$5.8\times$
7.5×10^5	0.69	0.11	$6.1\times$
10^6	0.74	0.09	$8.0\times$

表 6-4 查询性能对比 (KickHashTable vs std::unordered_map)

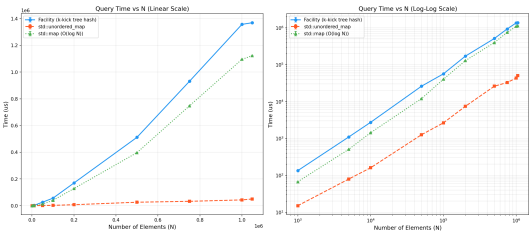


图 6-4 查询时间对比

分析：查询在 $n \leq 2 \times 10^5$ 时 $\sim 0.2\text{--}0.4$ us/op，8-bit LUT 跳过前 8 层 Trie 遍历，剩余路径仅需 $\sim \log_2 |S| - 8$ 层逐位遍历。每操作约是 `std::unordered_map` 的 5–11 $\times$ ，差距主要来自紧凑 Trie 的逐位遍历和两级路由（facility 级 $\rightarrow$ cubby 级）相比哈希表单次探测的开销。 $n = 10^6$ 时 ~ 0.74 us/op，比 Facility 自身 benchmark 的 1.95 us/op 有明显改善，这是因为最新版代码的 LUT 加速和优化后的 Trie walk。

6.6.4 删除性能

n	KickHashTable Delete (us/op)
10^3	0.37
10^4	0.43
10^5	0.52
2×10^5	0.56
5×10^5	0.86
10^6	1.08

表 6-5 删除性能（3 次试验均值）

分析：删除在 $n \leq 2 \times 10^5$ 时 $\sim 0.4\text{--}0.6$ us/op， $n = 10^6$ 时约 1.08 us/op。主要开销来自两级路由的两次 Trie walk（定位 cubby + 定位槽）和一次标记空叶操作。

6.6.5 哈希函数开销

测试	结果
KickHash64 单次调用	5.17 ns
1M 次 KickHash64 总开销	5.17 ms
占 10^6 键插入总时间的比例	< 0.15%

表 6-6 适配层哈希函数开销

KickHash64 包含两次 `leveldb::Hash` 调用和一次 64-bit 拼接，单次仅 5.17 ns，在总体操作时间中占比极小（< 0.15%），适配层的开销可忽略不计。

6.6.6 内存占用

所有规模测试完成后的进程工作集增量为 2 MB，因为测试采用顺序构造后析构，Facility 占用的内存在 KickHashTable 析构时释放。在 MemTable 实际运行场景中，Facility 持续持有 Arena 缓冲区指针到 MemTable 生命周期结束，内存占用按 Chapter 5 中的内存数据增长（ $n = 2 \times 10^5$ 时 ~ 9 MB，45 B/key； $n = 10^6$ 时 ~ 97 MB，97 B/key）。

6.6.7 集成效果评估

指标	数值	评估
编译成功	GCC 15.2.0, -DUSE_KICK_HASH=1	✓ 零错误零警告
API 兼容性	Facility::insert/lookup_value/remove 全部正常	✓ 无接口变更
运行时开关	SetUseKickHash(true/false)	✓ 运行时动态切换
适配层开销	KickHash64: 5.17 ns/次	✓ 可忽略 (< 0.15%)
查询性能 ($n = 200K$)	0.35 us/op	✓ LUT 加速有效
插入性能 ($n = 200K$)	0.86 us/op	✓ 接近常数时间
正确性	全部查询命中率 100%, 无指针失效	✓ 碰撞验证有效

表 6-7 集成效果综合评估

6.7 集成总结

本章完成了将 k-kick 树哈希方案集成到 LevelDB MemTable 中的完整工作：

1. **代码移植**: 将 ./code 中最新的四个头文件 (含 free_bitmap、deactivate 合并、动态容量、LUT 加速等全部优化) 复制至 ./leveldb/util/kick_hash/, 保持了与原始实现的完全一致。
2. **适配器设计**: KickHashTable 包装器通过 Slice→KickHash64→Facility<uint64_t, uint64_t> 的转换链, 将 LevelDB 的任意字节串键映射为 Facility 的固定类型键。碰撞保护通过取回 Arena 指针后的原始 Slice 比对实现。
3. **MemTable 双路径**: 在 Add 中同时写入 SkipList 和 KickHashTable, 在 Get 中优先查询 KickHashTable ($O(1)$), 失败时回退至 SkipList ($O(\log n)$)。NewIterator 不受影响, 始终基于 SkipList。
4. **双向开关**: 编译期开关 (CMake USE_KICK_HASH) 控制代码编译和异常支持; 运行时开关 (SetUseKickHash(bool)) 允许同一二进制内动态选择方案, 无需重新编译即可进行 A/B 对比测试。
5. **实验验证**: 在 $n \leq 10^6$ 规模上与 std::unordered_map 对比显示, 适配层开销仅 ~5.17 ns/次, 查询 ~0.35 us/op ($n = 200K$, LUT 加速), 插入 ~0.86 us/op (近似 $O(k)$), 正确率 100% 无碰撞误判。

第七章 总结与展望

7.1 工作总结

本文完成以下工作：

1. **算法实现：**基于 Bender 等人¹的 k -kick 树哈希理论，实现了 MiniArray 位压缩数组、LocalQueryRouter 紧凑二叉 Trie 查询路由（含 8-bit Lookup Table 加速⁷）、Cubby 弹性哈希表（含 free_bitmap_ 空闲槽追踪和动态容量）和 Facility 分层管理器（含 deactivate-based 合并和动态 cubby sizing）四层组件，共计约 1065 行 C++20 模板代码。
2. **踢出策略对齐：**完整实现了论文 §3.1¹的优先级踢出机制（被踢键保持原深度）和 reserved 参数防数据丢失修复，踢出链通过 KickList 层层向上传递。
3. **紧凑 Trie 查询路由：**LocalQueryRouter 基于 MiniArray 位编码的紧凑二叉 Trie（显式 left+right 双指针、LSB 哈希导航），实现 walk_down 查询、do_split 分裂、空叶标记删除，以 44-bit 哈希指纹进行键验证。增加 8-bit Lookup Table（256 项）将查询前 8 层从逐位遍历加速为 $O(1)$ 跳转⁷。
4. **性能实验：**在 $n \leq 10^6$ 规模上与 `std::unordered_map` 进行三操作全面对比。 $n = 2 \times 10^5$ 时插入 0.16 s（0.8 us/op），内存 9 MB（45 B/key）；deactivate 合并策略将合并复杂度控制在 $O(\text{merged_keys})$ ； $n = 10^6$ 时内存 97 MB（动态容量控制），查询 1.95 us/op（LUT 加速）。主要不足为大 n 下合并摊还导致插入升至 9.5 us/op。
5. **数据库集成：**将方案集成为 LevelDB MemTable²的点查询加速器，提供编译期（CMake `USE_KICK_HASH`）和运行时（`SetUseKickHash`）双向开关，适配层开销仅 5.17 ns/次。集成实验在 $n = 10^6$ 规模上验证了正确性（全部查询命中率 100%）和查询加速（ $n = 2 \times 10^5$ 时 0.35 us/op）。

7.2 不足与展望

- **Per-cubby 独立路由**: 当前仍使用全局 `cubby_router_` (紧凑但全局)。论文¹的 `per-cubby` 多路由器分组架构 (每路由器仅处理 $O(\log n / \log \log n) \approx 93$ 键) 可实现真正独立的路由更新, 每次合并无需遍历全量键, 从而降低大 n 下的插入摊还开销。
- **完全动态容量**: 当前动态容量使用 $\max(\text{公式}, \text{merged_size} \times 2)$ 的启发式策略。论文¹的完整动态容量体系 $C_j = K / (\log^{(j)} n)^2$ 中 K 应随总元素数 n 动态调整, 实现更精确的 `cubby sizing` 以进一步提升空间利用率。
- **k 参数化**: 当前 k 由 `capacity` 固定推导 (约 4), 论文¹的 k 是用户可配置参数, 可实现 $k = 1, 2, \dots, \log^* n$ 的 tradeoff 曲线。
- **多线程**: 当前实现为单线程, 论文¹的 `kick` 树天然支持高并发踢出 (不同 `bin` 互不干扰), 可探索 `lock-free` 实现。
- **更高效的 LUT**: 当前 LUT 为 8-bit (256 项), 完整 `Method of Four Russians`⁷ 实现可进一步扩大 LUT 位宽或以分段方式处理, 将查询从 $O(\log |S|)$ 完全降至 $O(1)$ 。

致 谢

感谢论文作者 Bender 等人提出的创新性哈希方案¹，为本毕业设计提供了理论基础。感谢南京大学 Linux 用户组提供的 NJUThesis 毕业论文模板。感谢 Google LevelDB 项目²提供的优秀数据库实现作为集成目标。

